Foundations of Computer Architecture & Organization

Instructor: Tek Raj Pant

March 24, 2026

Contents

1 Digital Logic and Data Representation **1**
1.1 The Binary Foundation . . . 1
1.1.1 Bits, Bytes, and Words . . . 1
1.1.2 Signed vs. Unsigned Integers . . . 1
1.2 Floating Point Representation . . . 2
1.3 Combinational Logic . . . 2
1.3.1 Boolean Algebra and Basic Gates . . . 2
1.3.2 Circuit Building Blocks . . . 4
1.4 Sequential Logic . . . 5
1.4.1 Latches and Flip-Flops . . . 5
1.4.2 System Clock . . . 5
1.4.3 For more details please watch this Computer Architecture from Scratch Video Series . . . 5

2 The Central Processing Unit (CPU) Internal Mechanics **6**
2.1 CPU Components . . . 6
2.1.1 ALU: How Operations Are Physically Gated . . . 6
2.1.2 Control Unit: Hardwired vs. Microprogrammed . . . 6
2.1.3 The Register File . . . 6
2.2 Critical Registers . . . 7

3 Instruction Set Architecture (ISA) **8**
3.1 ISA Classification: RISC vs. CISC . . . 8
3.2 Instruction Formats . . . 8
3.3 Addressing Modes . . . 8
3.4 Assembly Language Fundamentals . . . 9
3.4.1 For more details please watch this Video From Intel Technology, Part-1 . . . 9
3.4.2 For more details please watch this Video From Intel Technology, Part-2 . . . 9
3.4.3 Very Basics- How CPU Works . . . 9
3.4.4 Medium- How CPU Works . . . 9

4 The Memory Hierarchy **10**
4.1 The Speed-Capacity Gap . . . 10
4.2 Main Memory: SRAM vs. DRAM . . . 10
4.3 Cache Memory . . . 10
4.3.1 Cache Mapping Techniques . . . 11

4.3.2 Cache Write Policies 11
4.4 Virtual Memory 11
4.4.1 Watch this Youtube Video for Memory Hierarchy 11

5 Bus Systems and I/O Communication 12
5.1 System Buses 12
5.2 Bus Arbitration 12
5.3 Input/Output Mechanisms 12
5.3.1 Programmed I/O 12
5.3.2 Interrupt-Driven I/O 12
5.3.3 Direct Memory Access (DMA) 13
5.3.4 Watch this youtube Video for Address, Data and Control Buses 13
5.3.5 Watch this Youtube Video for I/O and Intruption 13

6 Pipelining and Performance 14
6.1 The Pipeline Concept 14
6.2 Pipeline Hazards 14
6.3 Instruction-Level Parallelism 15

7 How a Program Actually Works: The Execution Journey 16
7.1 The Compilation Phase 16
7.2 The Fetch Stage 16
7.3 The Decode Stage 17
7.4 The Execute Stage 17
7.4.1 The Complete Cycle 17
7.5 Function Call Example 17
7.6 Conclusion 17

A Further Reading and Resources 18

Index 19

Chapter 1

Digital Logic and Data Representation

1.1 The Binary Foundation

At the heart of every computer lies the binary system. Unlike humans who use ten digits (0–9), computers use only two digits: 0 and 1. These correspond to two voltage levels: typically 0 V (low) representing 0, and a positive voltage (e.g., 5 V or 3.3 V) representing 1. Each binary digit is called a **bit**.

1.1.1 Bits, Bytes, and Words

- **Bit**: The smallest unit of data, either 0 or 1.
- **Byte**: A group of 8 bits. It can represent 256 distinct values (2^8).
- **Word**: The natural unit of data that a processor handles. Typically 16, 32, or 64 bits. In modern systems, a *word* is often 32 bits, and a *double word* is 64 bits.

1.1.2 Signed vs. Unsigned Integers

Unsigned integers are straightforward: they represent non-negative numbers using standard binary. For example, an 8-bit unsigned integer can represent 0 to 255.

Signed integers require a method to represent negative numbers. The most common is **Two's Complement**.

- Positive numbers are represented as usual.
- To get the negative of a number, invert all bits and add 1.

For an 8-bit two's complement system, the range is -128 to $+127$. The most significant bit (MSB) indicates the sign: 0 for positive, 1 for negative.

Example 1.1. *Represent -5 in 8-bit two's complement:*

1. *$+5$ in binary: 00000101*
2. *Invert bits: 11111010*
3. *Add 1: 11111011*

So -5 is `11111011`.

1.2 Floating Point Representation

Integers are not sufficient for scientific or fractional numbers. The **IEEE 754** standard defines how real numbers are stored in binary. A single-precision (32-bit) floating-point number consists of three parts:

1. **Sign bit** (1 bit): 0 for positive, 1 for negative.
2. **Exponent** (8 bits): Stored with a bias of 127. The actual exponent = stored value 127.
3. **Mantissa (Significand)** (23 bits): The fractional part, with an implied leading 1 (normalized numbers).

Value $= (-1)^{\text{sign}} \times 1.\text{mantissa} \times 2^{\text{exponent}-127}$.

Example 1.2. *Represent* 6.625 *in single precision:*

- *Convert to binary:* $6.625_{10} = 110.101_2 = 1.10101 \times 2^2$.
- *Sign = 0.*
- *Exponent = 2 + 127 = 129 =* $\texttt{10000001}_2$.
- *Mantissa = 10101 (followed by zeros to fill 23 bits).*

Thus the 32-bit representation is: `0 10000001 10101000000000000000000`.

1.3 Combinational Logic

Digital circuits are built from logic gates. Combinational logic outputs depend only on the current inputs.

1.3.1 Boolean Algebra and Basic Gates

Digital circuits are built from logic gates. Combinational logic outputs depend only on the current inputs. The fundamental gates are described below.

- **AND**: Output is 1 only if all inputs are 1.
- **OR**: Output is 1 if at least one input is 1.
- **NOT**: Inverts the input.
- **XOR**: Output is 1 if inputs differ.
- **NAND, NOR**: Negated versions of AND and OR; they are universal gates (any logic function can be built using only NANDs or only NORs).

Truth Tables and Symbols For each gate, the truth table lists all possible input combinations and the resulting output. The standard ANSI/IEEE symbols are also described.

AND Gate

A	B	$A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

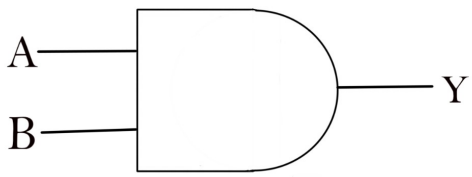


Figure 1.1: Logic AND Gate

OR Gate

A	B	$A + B$
0	0	0
0	1	1
1	0	1
1	1	1

Figure 1.2: OR Gate

NOT Gate (Inverter)

A	$\overline{A}$
0	1
1	0

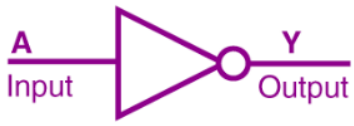


Figure 1.3: NOT Gate

XOR Gate (Exclusive OR)

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

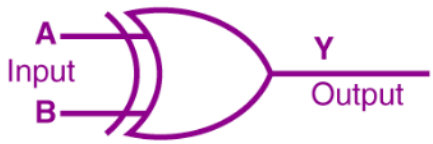


Figure 1.4: XOR Gate

NAND Gate (NOT AND)

A	B	$\overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

Table 1.1: NAND truth table

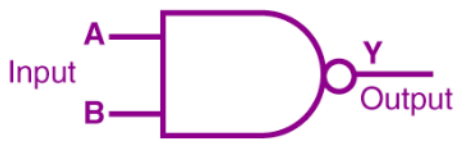


Figure 1.5: NAND Gate

NOR Gate (NOT OR)

A	B	$\overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

Figure 1.6: NOR Gate

These gates are the building blocks for all digital circuits, from simple adders to complex processors. Their behavior is described by Boolean algebra, which provides the mathematical foundation for logic design.

1.3.2 Circuit Building Blocks

Adders

A **half adder** adds two bits, producing a sum and a carry. A **full adder** adds three bits (two operands and a carry-in) and produces a sum and a carry-out. By cascading full adders, we build multi-bit adders.

Multiplexers (MUX)

A multiplexer selects one of several inputs based on select lines. For example, a 2-to-1 MUX has two data inputs, one select line, and one output. It is fundamental for routing data within the CPU.

Decoders

A decoder converts a binary code into a one-hot output. For instance, a 3-to-8 decoder takes a 3-bit address and asserts exactly one of eight output lines. Decoders are used in memory addressing and control logic.

Arithmetic Logic Unit (ALU)

The ALU performs arithmetic and logical operations. A simple ALU might support ADD, SUB, AND, OR, XOR, and NOT. It uses a multiplexer to choose the result of the desired operation based on control lines.

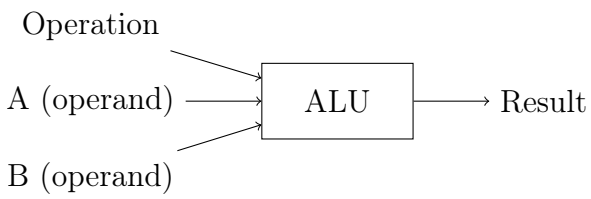


Figure 1.7: Conceptual ALU: takes two operands and an operation code, produces result.

1.4 Sequential Logic

Unlike combinational logic, sequential circuits have memory: their output depends on both current inputs and past state.

1.4.1 Latches and Flip-Flops

- **SR Latch**: The simplest memory element, set and reset inputs.
- **D Latch**: Captures the value on the D input when the enable (clock) is high.
- **Edge-triggered D Flip-Flop**: Captures D only on the rising (or falling) edge of the clock. This is the fundamental building block for registers.

1.4.2 System Clock

The clock is a square wave that synchronizes operations across the CPU. Each rising edge (or falling edge) triggers state updates. The clock frequency determines how many operations can be performed per second (e.g., 3 GHz = 3 billion cycles per second).

1.4.3 For more details please watch this Computer Architecture from Scratch Video Series

Chapter 2

The Central Processing Unit (CPU) Internal Mechanics

2.1 CPU Components

The CPU (Central Processing Unit) is the brain of the computer. Its main components are:

- **Arithmetic Logic Unit (ALU)**: Performs arithmetic and logical operations.
- **Control Unit (CU)**: Generates control signals to direct data flow and operations.
- **Register File**: A small, fast storage area for temporary data.

2.1.1 ALU: How Operations Are Physically Gated

The ALU contains circuits for addition, subtraction, AND, OR, etc. Control lines from the CU select which operation's result is passed to the output. For example, a 4-bit ALU may have a 3-bit control signal: 000 for ADD, 001 for SUB, 010 for AND, etc. The result is then stored in a register or fed back into the data path.

2.1.2 Control Unit: Hardwired vs. Microprogrammed

- **Hardwired Control**: The control signals are generated by combinational logic directly from the instruction opcode. This is fast but inflexible.
- **Microprogrammed Control**: Control signals are stored in a ROM (control store). Each machine instruction triggers a sequence of micro-instructions (microcode). This is more flexible and easier to modify.

2.1.3 The Register File

Registers are the fastest memory inside the CPU. They are typically divided into:

- **General-purpose registers**: Used for any data, e.g., R0, R1, ..., R15.
- **Special-purpose registers**: Dedicated functions (see below).

2.2 Critical Registers

Program Counter (PC) Holds the memory address of the next instruction to be executed.

Instruction Register (IR) Holds the current instruction being executed (the opcode and operands).

Memory Address Register (MAR) Holds the address of memory to read or write.

Memory Data Register (MDR) Holds the data being transferred to/from memory.

Stack Pointer (SP) Points to the top of the stack (used for function calls, local variables, and return addresses).

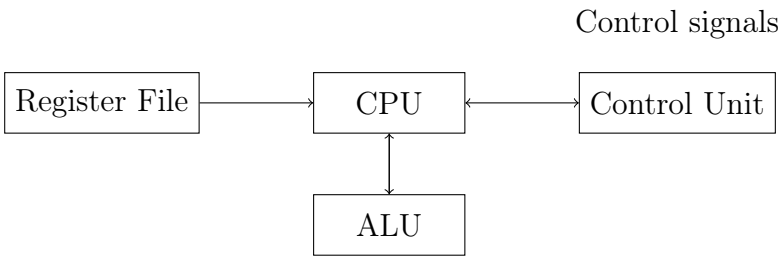


Figure 2.1: Simplified CPU data path.

Chapter 3

Instruction Set Architecture (ISA)

The ISA is the contract between hardware and software—it defines the instructions the CPU can execute.

3.1 ISA Classification: RISC vs. CISC

- **RISC (Reduced Instruction Set Computer)**: Few instructions, each one simple and typically one cycle. Emphasis on pipelining. Examples: ARM, RISC-V.
- **CISC (Complex Instruction Set Computer)**: Many instructions, some complex (e.g., string copy). Instructions can vary in length and cycles. Example: x86.

3.2 Instruction Formats

An instruction consists of an opcode (operation code) and zero or more operands. Formats can be:

- **Three-address**: `ADD R1, R2, R3` (R1 ← R2 + R3)
- **Two-address**: `ADD R1, R2` (R1 ← R1 + R2)
- **One-address (accumulator)**: `ADD X` (accumulator ← accumulator + X)
- **Zero-address (stack)**: operations use the top of stack implicitly.

3.3 Addressing Modes

Addressing modes specify how to compute the effective address of an operand.

Immediate Operand is in the instruction itself: `ADD R1, #5`

Direct Address of operand is given: `LOAD R1, 1000` (load from memory location 1000)

Indirect Address is stored in a register: `LOAD R1, (R2)` (R2 contains the address)

Indexed Base + index: `LOAD R1, 1000(R2)` (address = 1000 + R2)

Relative PC + offset: used for branching.

3.4 Assembly Language Fundamentals

Assembly language is a human-readable representation of machine code. Common instructions:

```
; Data movement
MOV  eax, 5          ; immediate to register
MOV  ebx, [1000]     ; direct memory to register
LOAD R1, R2          ; R1 <- R2 (typical RISC)
STORE R1, [R2]       ; store R1 to address in R2

; Arithmetic/Logic
ADD  eax, ebx        ; eax = eax + ebx
SUB  eax, 10
AND  ecx, edx
OR   ecx, edx

; Control flow
CMP  eax, ebx        ; compare, sets flags
JE   label           ; jump if equal
JMP  label           ; unconditional jump
CALL func            ; call subroutine
RET                  ; return from subroutine
```

Listing 3.1: Sample x86 assembly

3.4.1 For more details please watch this Video From Intel Technology, Part-1

3.4.2 For more details please watch this Video From Intel Technology, Part-2

3.4.3 Very Basics- How CPU Works

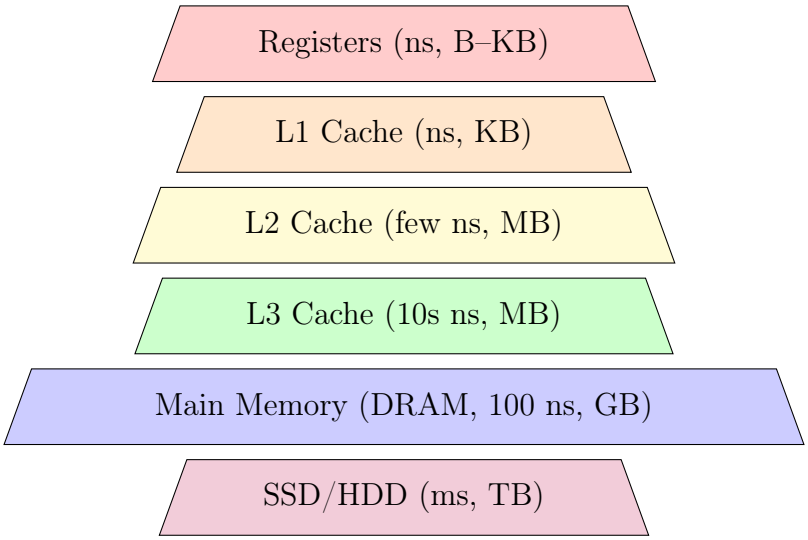
3.4.4 Medium- How CPU Works

Chapter 4

The Memory Hierarchy

4.1 The Speed-Capacity Gap

Memory technologies have a trade-off: faster memory is more expensive per byte and lower capacity. To balance speed and cost, computers use a hierarchy:



4.2 Main Memory: SRAM vs. DRAM

- **SRAM (Static RAM)**: Uses flip-flops. Fast, no refresh, but lower density and higher cost. Used for cache.
- **DRAM (Dynamic RAM)**: Uses capacitors. Slower, requires periodic refresh, but higher density and cheaper. Used for main memory.

4.3 Cache Memory

Cache is a small, fast memory that stores frequently used data to reduce average access time.

4.3.1 Cache Mapping Techniques

When the CPU requests data, the cache controller maps the main memory address to a cache location.

Direct-mapped Each memory block maps to exactly one cache line. Simple but prone to conflicts.

Fully associative Any block can go in any line; best hit rate but hardware complex.

Set-associative A compromise: cache is divided into sets, each set has n lines (e.g., 8-way set-associative).

4.3.2 Cache Write Policies

- **Write-through**: Write to cache and main memory simultaneously.
- **Write-back**: Write only to cache; mark line dirty; write to main memory when evicted.

4.4 Virtual Memory

Virtual memory allows programs to use more memory than physically available and isolates processes.

- **Paging**: Main memory is divided into fixed-size pages. Virtual addresses are translated to physical addresses via a page table.
- **Segmentation**: Variable-sized segments; less common today.
- **Translation Lookaside Buffer (TLB)**: A cache for page table entries to speed address translation.

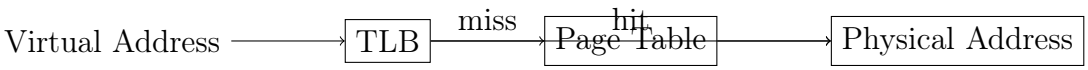


Figure 4.1: Virtual address translation using TLB and page table.

4.4.1 Watch this Youtube Video for Memory Hierarchy

Chapter 5

Bus Systems and I/O Communication

5.1 System Buses

A bus is a set of parallel wires that connect components. Three main buses:

- **Address bus**: Carries memory addresses from CPU to memory. Width determines maximum addressable memory (e.g., 32 bits → 4 GB).
- **Data bus**: Carries data between CPU, memory, and I/O. Width affects performance.
- **Control bus**: Carries control signals (read/write, interrupt, clock).

5.2 Bus Arbitration

When multiple devices want to use the bus, an arbiter decides who gets access. Common methods:

- **Daisy chain**: Priority based on position.
- **Centralized arbitration**: A bus controller grants requests.
- **Distributed arbitration**: Devices self-manage (e.g., Ethernet CSMA/CD).

5.3 Input/Output Mechanisms

5.3.1 Programmed I/O

The CPU actively polls devices to check if they are ready. Simple but wastes CPU cycles.

5.3.2 Interrupt-Driven I/O

A device sends an interrupt signal to the CPU, which suspends its current work, saves context, and runs an Interrupt Service Routine (ISR). After handling the interrupt, the CPU resumes.

5.3.3 Direct Memory Access (DMA)

A DMA controller transfers data between I/O devices and memory without CPU involvement. The CPU initiates the transfer and then continues other work. When the transfer is complete, the DMA controller interrupts the CPU.

5.3.4 Watch this youtube Video for Address, Data and Control Buses

5.3.5 Watch this Youtube Video for I/O and Intruption

Chapter 6

Pipelining and Performance

6.1 The Pipeline Concept

Pipelining divides instruction execution into stages. Typical stages:

1. Fetch (IF)
2. Decode (ID)
3. Execute (EX)
4. Memory Access (MEM)
5. Write-back (WB)

By overlapping stages, throughput increases: one instruction can complete per cycle (ideally).

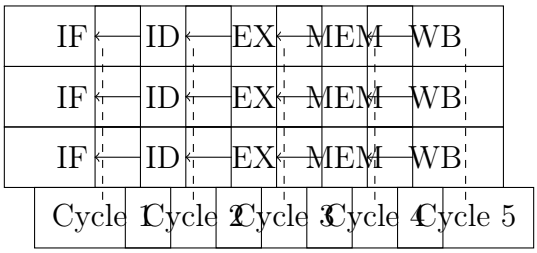


Figure 6.1: Pipelined execution of three instructions. Each instruction still takes 5 cycles, but throughput is 1 per cycle.

6.2 Pipeline Hazards

Hazards prevent the next instruction from executing in the next clock cycle.

Structural hazard Hardware resource conflict (e.g., one memory port for both instruction fetch and data access). Solved by adding more resources or stalling.

Data hazard Instruction depends on result of previous instruction that is not yet ready. Solved by forwarding (bypassing) or stalling.

Control hazard Branch instructions change the flow. Solved by branch prediction, delayed branching, or flushing.

6.3 Instruction-Level Parallelism

To improve performance beyond single-pipelined execution, modern CPUs use:

- **Superscalar architecture**: Multiple execution units allow issuing several instructions per cycle.
- **Out-of-order execution**: Instructions are scheduled dynamically to avoid stalls, as long as data dependencies are respected. Results are retired in program order to maintain correctness.

Chapter 7

How a Program Actually Works: The Execution Journey

To tie everything together, we trace the life of a simple program from source code to execution.

7.1 The Compilation Phase

Consider a high-level statement in C:

```c
int x, y;
x = y + 5;
```

The compiler transforms this into assembly:

```
; Assume y is at memory address 1000, x at 1004
LDR R1, [1000]     ; load y into register R1
ADD R1, R1, #5     ; add 5
STR R1, [1004]     ; store result into x
```

The assembler then converts each mnemonic into machine code (binary). For a RISC machine, these instructions are fixed length (e.g., 32 bits) and are placed in memory starting at some address, say 0x4000.

7.2 The Fetch Stage

1. The Program Counter (PC) initially contains 0x4000.
2. The CPU places 0x4000 onto the address bus.
3. Memory returns the binary instruction (e.g., 0xE5C10000) on the data bus.
4. This instruction is loaded into the Instruction Register (IR).
5. PC is incremented to 0x4004 (next instruction).

7.3 The Decode Stage

The Control Unit examines the opcode field of the instruction in the IR. It determines:

- This is a `LDR` (load) instruction.
- The source is memory address 1000, destination is R1.
- It asserts control signals: enable MAR to load address 1000, enable read on memory, set MUX to route data to R1, etc.

7.4 The Execute Stage

For a load instruction, the "execute" stage is usually the memory access:

- MAR receives 1000.
- Memory returns the value at 1000 into MDR.
- MDR is written into register R1.

For the subsequent `ADD` and `STR`, the ALU performs the addition, and the final store writes the result back to memory.

7.4.1 The Complete Cycle

The CPU repeats this fetch-decode-execute cycle billions of times per second. The PC moves through instructions linearly until a branch or jump modifies it.

7.5 Function Call Example

When a function is called (e.g., using `CALL` in assembly), the CPU:

1. Pushes the return address (current PC) onto the stack.
2. Sets PC to the address of the function.
3. The function's code uses the stack for local variables (via the Stack Pointer).
4. On return (`RET`), the return address is popped from the stack back into PC.

7.6 Conclusion

From the humble logic gate to the complex pipeline, every layer of the computer works in harmony to execute the programs we write. Understanding this hierarchy—binary representation, CPU design, memory hierarchy, and pipelining—gives us insight into performance, efficiency, and the fundamental limits of computing.

Appendix A

Further Reading and Resources

- *Computer Organization and Design* by Patterson & Hennessy.
- *Digital Design and Computer Architecture* by Harris & Harris.
- Online resources: RISC-V specifications, ARM Architecture Reference Manual.

Index